

Python

Summary Session 1



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Что такое Python?

?

Что такое компилятор и интерпретатор?

?

Из каких элементов состоит машинный код?

Полезно знать



Python

Это высокоуровневый язык программирования, известный своей простотой и читабельностью.



Простота



Динамическая типизация



Интерпретируемый



Кроссплатформенность



Сообщество и экосистема

Интерпретатор и компилятор



Компилятор

Программа, которая сначала переводит весь исходный код в машинный код, а затем запускает его.

Компилятор преобразует весь код сразу.

Интерпретатор

Программа, которая выполняет код построчно, сразу переводя его в машинный код.

Интерпретатор исполняет код по мере чтения.

Python — интерпретируемый язык



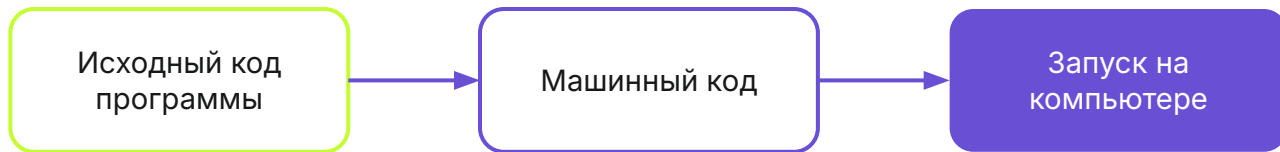
Позволяет запускать программы без предварительной компиляции.
Это ускоряет разработку и упрощает отладку, но может быть медленнее по сравнению с компилируемыми языками.



Машинный код

Это низкоуровневые инструкции, которые процессор компьютера может напрямую выполнять.

Машинный код



Экспресс-опрос

?

Что такое Jupyter Notebook?

?

Какими преимуществами обладает Jupyter Notebook?

?

Приведите пример переменной.



Jupyter Notebook

Это интерактивная среда разработки и обмена документами, которая позволяет создавать и выполнять код Python в виде ячеек.

Jupyter Notebook



Состоит из веб-интерфейса, который позволяет создавать и редактировать ноутбуки, а также встроенного интерпретатора Python, который выполняет код в каждой ячейке ноутбука. Каждая ячейка может содержать код, текстовые описания, формулы, изображения, графики и другие элементы.

Jupyter Notebook



Преимущества

Разработчик может выполнять код в ячейках по одной или несколько сразу, что позволяет получать немедленные результаты и видеть вывод прямо в ноутбук.



Интерактивность



Удобно для тестирования
и экспериментирования с кодом

Полезно знать



Переменная

- Это именованная область памяти, в которой хранятся данные.
- После создания переменной можно получить доступ к хранящимся в ней данным по заданному имени.

Данные

- Это информация, которая может быть обработана и сохранена компьютером.
- В программировании данные представляют собой любые значения, которые программа использует для выполнения операций с ними.

Пример



Целые числа: **5, -3, 42**



Числа с плавающей запятой или вещественные: **3.14, -0.001, 2.0**



Строки: **"Привет", 'Мир'**



Данные

Данные могут поступать из разных источников, таких как файлы, базы данных, пользовательский ввод или внешние устройства.

Программы обрабатывают эти данные, изменяют их, анализируют или сохраняют для дальнейшего использования.

Экспресс-опрос

?

Какое определение у оператора присваивания?

?

Приведите пример оператора присваивания.

Оператор присваивания



Определение

- Это символ, который используется для присвоения значения переменной.
- В Python оператор присваивания обозначается знаком `=`. Он связывает переменную с заданным значением.

Пример

```
x = 10 # Присваиваем переменной x число 10
name = "John" # Присваиваем переменной name строку "John"
```

Экспресс-опрос

?

Какие существуют правила чистого кода?

?

Приведите пример правильного использования наименования переменных.

Правила для чистого и понятного кода:

-  Имя переменной может содержать только буквы, цифры и знак подчеркивания _
-  Начинается только с буквы или знака подчеркивания. (1number не подходит)
-  Переменные регистрозависимы ("age" и "Age" - разные переменные)
-  Не используйте зарезервированные слова (как if, else, while и т.д.)
-  Не используйте названия функций (как print, int, max и т.д.)

Правила для чистого и понятного кода:



В Python переменные принято записывать в формате snake_case



По имени переменной должно быть понятно что в ней хранится



Имя переменной должно быть достаточно длинным, чтобы передавать смысл



Но не слишком длинным. Это улучшает читаемость

Правила именования переменных



Правильные имена переменных:

`coordinate1 = 42`

`user_name = "Alice"`

`_price = 10`

Неправильные имена переменных:

`2nd_coordinate = 33` # Начинается с цифры

`my-variable = 10` # Содержит дефис

`my variable = 10` # Содержит пробел

`total%items = 5` # Содержит недопустимый символ "%"

`if = 42` # Использует зарезервированное слово "if"

Экспресс-опрос

?

Что такое синтаксис в языке программирования?

?

Какие существуют правила синтаксиса в Python?

?

Приведите пример ошибки в синтаксисе.



Синтаксис

Это набор правил, определяющих, как должны быть организованы конструкции в языке программирования.

Синтаксис в Python включает:



Структуру кода



Идентификаторы



Строки и комментарии



Операторы

Некоторые правила синтаксиса



Конец строки

Конец строки является концом инструкции.

Пример

```
num1 = 5  
num2 = 7
```

Некоторые правила синтаксиса



В одной строке

Иногда возможно записать несколько инструкций в одной строке, разделяя их точкой с запятой.

Однако это не рекомендуется.

Пример

```
num1 = 5; num2 = 7
```



Ошибки синтаксиса

(`SyntaxError`) — это ошибки, возникающие когда код не соответствует правилам языка. Эти ошибки препятствуют интерпретатору понять и выполнить код.

Ошибки синтаксиса



При запуске программы с ошибкой синтаксиса интерпретатор Python остановит выполнение кода и выведет сообщение об ошибке.

Сообщение об ошибке включает:



Тип ошибки



Описание ошибки



Номер строки



Строка кода

Рассмотрим ошибку



Код

```
1number = 5
```

Решение

При попытке запустить программу вы получите сообщение об ошибке

Недопустимые символы

```
1number = 5
```

```
Cell In[9], line 1
```

```
1number = 5
```

```
^
```

```
SyntaxError: invalid decimal literal
```

Неправильное использование кавычек для строк

```
name = "John'
```

```
Cell In[10], line 1
```

```
    name = "John'
```

```
        ^
```

```
SyntaxError: unterminated string literal (detected at line 1)
```

Неправильное использование операторов

```
1+*2
```

```
Cell In[2], line 1
```

```
1+*2
```

```
^
```

```
SyntaxError: invalid syntax
```

Экспресс-опрос

?

Что такое комментарии?

?

Какие существуют правила при использовании комментариев?



Комментарии

используются в Python для пояснения кода и делают его более понятным. Они не влияют на выполнение программы, так как интерпретатор игнорирует их.

Основные правила



В одной строке

- Начинаются с символа # и продолжаются до конца строки.
- Используются для кратких пояснений.

Пример

Это комментарий

x = 5 # Присваиваем значение 5 переменной x

Экспресс-опрос

?

Что такое PEP 8?

?

Какие существуют особые требования к финансовым сервисам?



PEP 8 (Python Enhancement Proposal 8)

Это документ, который содержит рекомендации по стилю написания кода на Python. Его цель — улучшить читаемость кода.

PEP 8



Соблюдение PEP 8 помогает разработчикам писать чистый и понятный код, который легче поддерживать и развивать.

Основные рекомендации PEP 8:



Длина строки



Пробелы



Именованние



Комментарии

Экспресс-опрос

?

Что такое функция?

?

Что такое вызов функции?

?

Какие действия необходимы чтобы вызвать функцию?



Функция

Это заранее написанный блок кода, который выполняет определённую задачу. В Python есть встроенные функции, которые можно использовать, просто вызвав их по имени.



Вызов функции

Это процесс выполнения функции в коде.

Чтобы вызвать функцию, нужно:



Указать её имя



Указать круглые скобки сразу после имени (они инициируют выполнение функции)



Передать необходимые данные (аргументы) в круглых скобках, при необходимости

Вызов функции



Код

```
print("Привет, мир!")
```

Разбор

print — это имя функции.

() — вызов функции.

"Привет, мир!" — это аргумент, который передаётся функции.

Экспресс-опрос

?

За что отвечает функция `print()`?

?

Приведите пример синтаксиса функции `print()`?



Функция print

Это встроенная функция в Python, которая выводит информацию на экран (в консоль). Она может выводить текст, числа, переменные и другие типы данных.

Пример: `print("Привет, мир!")`

```
print("Привет, мир!")
```

Привет, мир!

Пример: `print("Число:", 10)`

```
print("Число:", 10)
```

Число: 10

Экспресс-опрос

?

За что отвечает функция `input()`?

?

Приведите пример синтаксиса функции `input()`?



Функция input

Это встроенная функция в Python, которая позволяет получать данные от пользователя через ввод с клавиатуры.

Функция input



Когда вызывается `input()`, программа приостанавливается и ждёт, пока пользователь введёт текст и нажмёт Enter. На месте вызова функции появляется возвращаемое значение, которое далее можно присвоить переменной или передать в другую функцию.

Пример использования



Код

```
name = input("Введите ваше имя: ")  
print("Привет,", name)
```

Разбор

- Функция input выводит строку "Введите ваше имя: " и ждет ввода от пользователя.
- Введённый текст сохраняется в переменной name.
- С помощью print выводится приветствие с именем пользователя.

Экспресс-опрос

?

Что такое объект в Python?

?

Что такое тип данных?

?

Чему служит динамическая типизация?



Объект в Python

Это любое значение, с которым можно работать в программе.

Полезно знать



Объект

Всё в Python — это объект: числа, строки, списки и т.д.

Хранение

Каждый объект хранит данные и имеет определённый тип.



Тип данных

Определяет, какие значения может хранить объект и какие операции можно выполнять с этими значениями.



Динамическая типизация

Python является динамически типизированным языком, что означает, что тип переменной определяется автоматически в момент присваивания значения, и он может меняться в процессе выполнения программы.

Полезно знать



Переменная

Тип данных присваивается не переменной, а объекту, на который она ссылается. Это позволяет одной и той же переменной в ходе программы ссылаться на объекты разных типов данных.

Код

`price = 10` # Тип автоматически определен как целое число

`product = "smartphone"` # Тип автоматически переопределен как строка

Экспресс-опрос

?

Какие существуют примитивные типы данных?

?

Чем целые числа отличаются от чисел с плавающей запятой?

?

Какие значения может принимать тип данных bool?



Примитивные типы данных

Это основные типы, которые используются для хранения простых (единичных) значений.

Примитивные типы данных:



Целые числа (int)



Числа с плавающей запятой (float)



Строки (str)



Логические значения (bool)



Отсутствие значения (NoneType)

Целые числа (int)



Определение

Это значения без дробной части. Они могут быть положительными, отрицательными или равны нулю.

Пример

x = 5 # Положительное целое число
y = -3 # Отрицательное целое число
z = 0 # Ноль

Числа с плавающей запятой (float)



Определение

Это значения с дробной частью.

Пример

```
pi = 3.14          # Число с плавающей запятой  
temperature = -5.2 # Отрицательное число с  
плавающей запятой
```

Строки (str)



Определение

Это упорядоченные последовательности символов, заключённых в одинарные или двойные кавычки, а также в тройные кавычки (как одинарные, так и двойные).

Пример

```
# Одинарные кавычки
single_quote_string = 'Это строка в одинарных
кавычках.'
```

```
# Двойные кавычки
double_quote_string = "Это строка в двойных
кавычках."
```

Строки (str)



Определение

Тройные кавычки позволяют создавать многострочные строки, сохраняющие исходное форматирование, включая переносы строк.

Пример

```
# Одинарные тройные кавычки (многострочная строка)
triple_quote_string = "Это строка в
тройных кавычках, которая
может занимать несколько строк."
```

Логические значения (bool)



Определение

Логические значения представляют два возможных состояния: True (истина) и False (ложь). Они записываются текстом с большой буквы без кавычек.

Пример

```
is_student = True      # Логическое значение True
has_passed = False     # Логическое значение
False
```

Отсутствие значения (NoneType)



Определение

Тип NoneType представляет отсутствие значения. Единственное значение для этого типа: None

Пример

```
value = None # Переменная, не
              содержит никакого значения
```



Отсутствие значения

Значение None часто используется для инициализации переменных, когда значение ещё не известно, или для обозначения того, что функция ничего не возвращает.

Экспресс-опрос

?

Приведите пример использования возвращаемого значения

?

Что такое процедуры в Python?



Возвращаемое значение

Это результат, который функция передаёт обратно в место своего вызова.

Возвращаемое значение



Определение

Функции могут возвращать значение, или ничего не возвращать, в последнем случае по умолчанию будет возвращено None.

Результат, который вернула функция, можно присвоить некоторой переменной.

Пример

```
y = print("Привет, мир!")  
x = input()
```

Привет, мир!

x, y

('45', None)



Процедуры в Python

В Python нет отдельного понятия "процедура", как в некоторых других языках программирования.

Процедуры в Python



Определение

Термин процедура часто используется для описания функций, которые ничего не возвращают (или возвращают None) и выполняют определённые действия, такие как вывод на экран или модификация данных.

Пример

```
print("Привет, мир!") # Выводим сообщение, но  
print не возвращает результат
```

Экспресс-опрос

?

Чему служит передача аргументов в функцию?

?

Приведите пример передачи аргументов в функцию.



Передача аргументов в функцию

В функцию можно передавать аргументы
разными способами.

Передача аргументов в функцию



Способ 1

Значения (литералы), например, числа или строки.

Пример

```
print(5)  
print("Привет, мир!")
```

Передача аргументов в функцию



Способ 2

Переменные, из которых будут получены значения.

Пример

```
text = "Привет, мир!"  
print(text)
```

Передача аргументов в функцию



Способ 3

Результат выполнения математических операций.

Пример

```
print(2 + 5)
```

Передача аргументов в функцию



Способ 4

Результаты выполнения других функций.

Пример

```
print(input("Введите имя: "))
```

input возвращает то, что ввел пользователь.

Это значение можно передать в функцию print.

Экспресс-опрос

?

За что отвечает функция `type()`

?

Приведите пример использования функции `type`.



Функция type

Используется для определения типа объекта.
Она возвращает класс типа объекта.

Функция type



Определение

Она возвращает тип данных (класс), который находится в переменной в момент выполнения функции type.

Чтобы увидеть результат вызова функции type, необходимо передать его как аргумент в функцию print.

Пример

```
x = 10
print(type(x)) # <class 'int'>
x = "Привет"
print(type(x)) # <class 'str'>
```

Экспресс-опрос

?

Какие существуют строковые операции?

?

Что такое конкатенация?

Строковые операции:



Конкатенация



Умножение строк

Конкатенация



Определение

Конкатенация — это процесс объединения двух или более строк в одну строку. Для этого используется оператор +.

Пример

```
str1 = "Привет"
str2 = "мир"
result = str1 + " " + str2 # Объединение строк с
                             пробелом между ними
print(result) # Вывод: Привет мир
```



Конкатенация

Позволяет создавать новые строки путём добавления одной строки к другой. Это может быть полезно для комбинирования данных или формирования строки для вывода из нескольких элементов.

Умножение строк



Определение

Умножение строк позволяет создать новую строку, состоящую из повторений исходной строки.

Это делается с использованием оператора `*`, за которым следует число, указывающее, сколько раз строка должна быть повторена.

Пример

```
str1 = "Hi! "  
result = str1 * 3 # Создание строки из трёх  
повторений исходной строки  
print(result) # Вывод: Hi! Hi! Hi!
```



Умножение строк

Умножение может быть полезно для создания разделителей, повторяющихся шаблонов текста и других целей.

Обратите внимание, что при использовании этих операций исходные строки не изменяются.

Экспресс-опрос

?

Как в Python используются кавычки?

?

Чем отличаются одинарные и двойные кавычки?



Кавычки в Python

В Python строки можно заключать в одинарные ('') или двойные кавычки (""), что позволяет гибко работать с текстом.

Одинарные и двойные кавычки



Рекомендация

Если внутри строки необходимо использовать кавычки, то целесообразно выбирать противоположные по типу кавычки для оформления строки.

Пример

Одинарные кавычки внутри строки
quote1 = "Она сказала: 'Привет!'"

Двойные кавычки внутри строки
quote2 = 'Он ответил: "Привет!"'

Многострочные строки



Рекомендация

Для многострочных строк и строк, содержащих как одинарные, так и двойные кавычки, удобно использовать тройные кавычки (''' или ''').

В данном случае внутри строки можно использовать как одинарные, так и двойные кавычки.

Пример

```
multi_line_string = """Это многострочная строка, в  
которой есть 'одинарные' и "двойные" кавычки."""
```

Экспресс-опрос

?

Что такое экранирование?

?

Что такое Escape-последовательности?

?

Для чего применяется экранирование обратного слэша?



Экранирование

Это способ обработки специальных символов внутри строк.

В Python используется обратный слэш (\) для обозначения специальных символов и создания **escape-последовательностей**.



Escape-последовательности

Представляют собой комбинации символов для корректного отображения таких символов, как кавычки, обратный слэш, перенос строки или табуляцию, позволяя включать их в строку без конфликтов с синтаксическими правилами.

Экранирование кавычек



Рекомендация

Если строка обрамлена в одинарные кавычки, а внутри нужно использовать одинарную кавычку, её нужно экранировать. То же правило действует и для двойных кавычек.

Пример

```
# Одинарные кавычки внутри строки
string1 = 'В строке есть \'одинарные\' и "двойные" кавычки.'
print(string1)

# Двойные кавычки внутри строки
string2 = "В строке есть 'одинарные' и \"двойные\" кавычки."
print(string2)
```

Специальные символы



Рекомендация

Экранирование также используется для вставки специальных символов, таких как:

`\n` — новая строка

`\t` — табуляция

Пример

Новая строка

```
text = "Первая строка\nВторая строка"
```

```
print(text)
```

Табуляция

```
text_with_tab = "Первая строка\tВторая строка"
```

```
print(text_with_tab)
```

Экранирование обратного слэша



Рекомендация

Если в строке необходимо вставить обратный слэш, его тоже нужно экранировать, так как сам обратный слэш является специальным символом.

Пример

```
path = "C:\\Users\\Username\\Documents"
```

Экспресс-опрос

?

Как используются параметры `sep` и `end`?

?

Приведите примеры использования параметров `sep` и `end`.



Параметры `sep` и `end`

Функция `print` имеет несколько необязательных параметров, которые позволяют настроить ее поведение.

Параметры sep и end



sep (разделитель)

Определяет строку, которая будет вставлена между аргументами. По умолчанию используется пробел.

```
print("one", "two", "three")           # one two three
print("one", "two", "three", sep="--")  #
one--two--three
```

end (конец)

Определяет строку, которая будет добавлена в конец вывода. По умолчанию используется символ новой строки \n.

```
print("Привет", "мир", end="!!")
print("Привет")    # Привет, мир!!Привет
```



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Разбор домашнего задания

1. Приветственное сообщение

Создайте программу, которая выведет строки "Hi!" и "Hello!" на разных строках.

Пример вывода:

Hi!

Hello!

Решение:

```
print("Hi!")
```

```
print("Hello!" )
```

Разбор домашнего задания

2. Именное приветствие

Создайте программу, которая попросит пользователя ввести его имя, а затем выведет фразу: "Hello, <Name>" на новой строке.

Пример вывода:

```
Hello, Name
```

Решение:

```
name = input("Введите ваше имя: ")  
print("Hello, ", name)
```

Разбор домашнего задания

1. Числовая цепочка

Напишите программу, которая выводит числа от 1 до 5 включительно, разделяя их символами "---". Используйте параметр `sep`.

Пример вывода:

1---2---3---4---5

Решение:

```
print(1, 2, 3, 4, 5, sep="---")
```

Разбор домашнего задания

2. Пробельные символы

Напишите программу, которая выведет две строки: первая строка с табуляцией, вторая — на новой строке с помощью одного вызова `print()`.

Пример вывода:

Первая строка с табуляцией

Вторая строка на новой строке

Решение:

```
print("Первая строка\tс табуляцией\nВторая строка на новой строке")
```

Разбор домашнего задания

3. Текст в кавычках

Напишите программу, которая выводит строку "Это файл `example.txt`". Выполните задачу несколькими способами.

Пример вывода:

Это файл `example.txt`

Решение:

```
print("Это файл \"example.txt\"")
```

```
print('Это файл "example.txt"')
```

```
print('\'Это файл "example.txt"\'')
```



ВОПРОСЫ



Заключение

